// GUIDA GRATIS

# LA CARTELLA CHE FERMA CLAUDE *dal rompere tutto.*

Quando chiedi una modifica a Claude, a volte rompe in silenzio qualcosa che funzionava, e paghi due volte per sistemarla. Questa cartella open source lo impedisce con 4 passaggi sempre uguali. Qui sotto, uno per uno.

| | | |
|---|---|---|
| **1 · Copia** | lavora su una copia, mai sulla versione viva | passo 01 |
| **2 · Costruisci** | la modifica in un posto solo | passo 02 |
| **3 · Prova** | registra lo schermo mentre testa | passo 03 |
| **4 · Review** | una seconda AI rivede il lavoro | passo 04 |

// DOVE SI TROVA

`github.com/michaelshimeles/skills · 1.360 stelle (verificate il 20/10/2026) · per Claude Code, Cursor e Codex`

// COSA TROVI QUI DENTRO

Per ogni passaggio: **cosa fa** la skill, **cosa serve** per usarla e **dove resti tu a decidere**. In fondo l'installazione e i limiti onesti.

// PASSO 01 · COPIA

# 01 MAI SULLA VERSIONE *che usi davvero.*

La prima skill si chiama **new-feature**. Prima di toccare qualunque cosa, Claude crea una copia di lavoro (un «worktree» Git) partendo da origin/main.

**01 Parte da una copia pulita**
Ogni modifica nasce in un worktree separato, basato sulla versione principale. La versione su cui gira il tuo business resta com'è.

**02 Una copia per ogni lavoro**
Se due sessioni lavorano sullo stesso progetto, ognuna ha la sua copia: non si pestano i piedi.

**03 Controlla i conflitti prima**
Se altre modifiche in corso toccano le stesse cose, lo verifica; se sono indipendenti, procede.

**04 Pulizia sicura**
Non cancella il lavoro di altri e riusa una copia già assegnata a quel compito, invece di crearne una nuova.

| | | |
|---|---|---|
| **Skill** | new-feature | cartella |
| **Base** | origin/main (versione principale) | intatta |
| **Se leggi soltanto** | nessuna copia necessaria | nessun costo |

// ESEMPIO PRATICO

Chiedi «aggiungi il campo telefono al modulo prenotazioni». Claude lavora nella copia: se qualcosa va storto, **butti la copia e il gestionale vero non si è mai mosso**.

// PASSO 02 · COSTRUISCI

# 02 UNA MODIFICA, *un posto solo.*

La seconda skill è **code-structure**. Il motivo per cui un bug «torna»: la stessa logica copiata in cinque punti. Qui si scrive una volta e si riusa.

01 **Azioni e servizi**
Le azioni coordinano il flusso; i servizi contengono le operazioni riutilizzabili. Se il progetto ha già una sua convenzione, la rispetta.

02 **Niente logica duplicata**
Se più flussi fanno la stessa cosa (es. invio email), la sposta in un servizio condiviso.

03 **Sistemi una volta, vale ovunque**
Un bug corretto nel servizio si corregge in tutti i flussi che lo usano, senza caccia ai doppioni.

04 **Non cambia le regole del gioco**
Mantiene invariati i confini di salvataggio e le transazioni già esistenti: sposta, non reinventa.

| | | |
|---|---|---|
| **Skill** | code-structure | `cartella` |
| **Prima** | la stessa logica in 5 punti | `da evitare` |
| **Dopo** | un servizio, 5 flussi che lo usano | `un fix = ovunque` |

// DOVE DECIDI TU

Il file **AGENTS.md** è il «regolamento»: decide **quali passaggi servono per quel lavoro**. Una modifica solo di testo non richiede tutto il giro.

// PASSO 03 · PROVA

# 03 UN VIDEO *non può mentire.*

La terza skill è **evidence-driven-testing**. Mentre Claude prova la modifica come farebbe un utente, un registratore filma lo schermo. Tu non leggi il codice: guardi la registrazione.

01 **Registra mentre testa**
Un registratore (scripts/evidence.py, Python 3 + FFmpeg) cattura la sessione. Comandi: doctor, start, annotate, stop.

02 **Annota ogni passaggio**
Ogni verifica viene segnata con l'orario e scritta sul video: sai cosa è stato provato e quando.

03 **Salva le prove**
Alla fine ottieni evidence.mp4, più un report.md e un manifest.json con il riepilogo dei test.

04 **Prima e dopo, a colpo d'occhio**
La skill before-and-after cattura screenshot prima/dopo e crea una tabella «Before I After» pronta da incollare.

| | | |
|---|---|---|
| **Serve** | Python 3 + FFmpeg (con libx264) | requisito |
| **Controllo** | python3 scripts/evidence.py doctor | verifica |
| **Output** | evidence.mp4 · report.md · manifest.json | prova |

// IL PUNTO CHIAVE

Se non è un'interfaccia, la prova cambia forma: numeri misurati, coppie input/output, estratti di log. **Nessuna modifica passa «perché sembra a posto».**

// PASSO 04 · REVIEW

# 04 UNA SECONDA AI *rivede tutto.*

La skill **greploop** manda il lavoro a Greptile, un servizio di code review con AI, e lo rimanda a Claude finché non resta niente da sistemare.

01 **Obiettivo: review pulita**
Cerca una review aggiornata 5/5, senza segnalazioni ancora aperte sulla versione corrente.

02 **Fino a 10 giri**
Il limite predefinito è 10 iterazioni (--max-iterations). Arrivato al tetto, si ferma e ti dice cosa resta.

03 **Installa in un comando**
npx skills add michaelshimeles/skills. Per l'ingresso alternativo: cp -r greploop greploop-apps ~/.claude/skills/

04 **Limiti onesti**
Greptile deve essere attivo sul tuo repository (servizio a parte) e serve la CLI gh autenticata. Nel repo le cartelle sono 7: oltre alle 6 del reel c'è unslop (pulisce i testi).

| | | |
|---|---|---|
| **Skill** | greploop + greploop-apps | `review` |
| **Requisiti** | Greptile sul repo · gh autenticata | `tuoi` |
| **Promesse** | nessun numero di bug o ore risparmiate | `dipende da te` |

// VUOI APPLICARLO ALLA TUA AZIENDA?

Costruire un sistema che lavora da solo e lascia a te solo l'approvazione è quello che facciamo per le aziende. **Rispondi a questo DM**: ti diciamo cosa conviene automatizzare per primo.